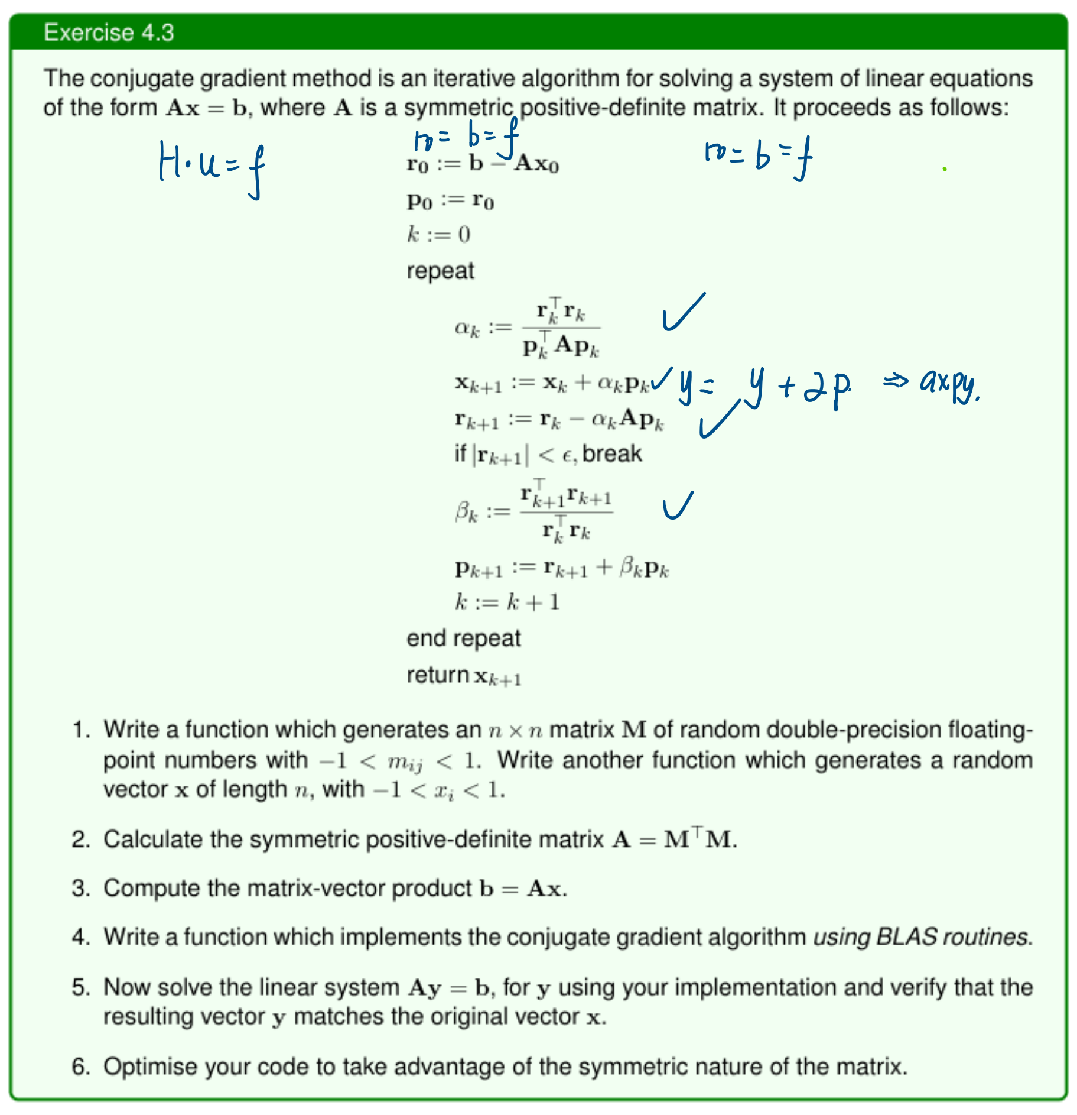
Ex 4-3 symmetric

1/30/24 08:02



Exercise 4.3

The conjugate gradient method is an iterative algorithm for solving a system of linear equations of the form $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is a symmetric positive-definite matrix. It proceeds as follows:

```
$\mathbf{r}_0 := \mathbf{b} - \mathbf{A}\mathbf{x}_0$
$\mathbf{p}_0 := \mathbf{r}_0$
$k := 0$
repeat
    $\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A}\mathbf{p}_k}$
    $\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$
    $\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A}\mathbf{p}_k$
    if $|\mathbf{r}_{k+1}| < \epsilon$, break
    $\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$
    $\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$
    $k := k + 1$
end repeat
return $\mathbf{x}_{k+1}$
```

1. Write a function which generates an $n \times n$ matrix $\mathbf{M}$ of random double-precision floating-point numbers with $-1 < m_{ij} < 1$. Write another function which generates a random vector $\mathbf{x}$ of length n, with $-1 < x_i < 1$.
2. Calculate the symmetric positive-definite matrix $\mathbf{A} = \mathbf{M}^\top\mathbf{M}$.
3. Compute the matrix-vector product $\mathbf{b} = \mathbf{Ax}$.
4. Write a function which implements the conjugate gradient algorithm *using BLAS routines*.
5. Now solve the linear system $\mathbf{Ay} = \mathbf{b}$, for $\mathbf{y}$ using your implementation and verify that the resulting vector $\mathbf{y}$ matches the original vector $\mathbf{x}$.
6. Optimise your code to take advantage of the symmetric nature of the matrix.

$H \cdot u = f$

$r_0 = b = f$

$r_0 = b = f$

✓

✓ $y = y + \alpha p$ ⇒ axpy.

✓

✓

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;
#include "cblas.h"
// Generate a random n x n matrix
void FillMatrix(int n, double* H) {
    for (int i = 0; i < n; ++i) {            // index rows
        for (int j = 0; j < n; ++j) {      // index cols
            H[i*n+j] = double(rand())/(RAND_MAX/2.0)-1.0;
        }
    }
}
// Generate a random vector
void FillVector(int n, double* f) {
    for (int i = 0; i < n; ++i) {
        f[i] = double(rand())/(RAND_MAX/2.0)-1.0;
    }
}
// Print a matrix
void PrintMatrix(int n, double* H) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cout << setw(10) << H[i*n+j] << " ";
        }
        cout << endl;
    }
}
// Print a vector
void PrintVector(int n, double* u) {
    for (int i = 0; i < n; ++i) {
        cout << u[i] << endl;
    }
}
// Solve the linear system
void SolveConjugateGradient(int n, double* A, double* b, double* x)
{
    double* r = new double[n];
    double* p = new double[n];
    double* t = new double[n]; //temp
    int k;
    double alpha;
    double beta;
    double eps;
    double tol = 0.00001;
    cblas_dcopy(n, b, 1, r, 1);        // r_0 = b (i.e. b)
    cblas_dsymv(CblasRowMajor, CblasUpper, n, -1.0, A, n,
                x, 1, 1.0, r, 1);  // r_0 = b - A x_0
    cblas_dcopy(n, r, 1, p, 1);        // p_0 = r_0
    k = 0;
    do {
        cblas_dsymv(CblasRowMajor, CblasUpper, n, 1.0, A, n,
                    p, 1, 0.0, t, 1);       // t = A p_k
        alpha = cblas_ddot(n, t, 1, p, 1);  // alpha = p_k^T A p_k
        alpha = cblas_ddot(n, r, 1, r, 1) / alpha; // compute
alpha_k
        beta  = cblas_ddot(n, r, 1, r, 1);  // r_k^T r_k
        cblas_daxpy(n, alpha, p, 1, x, 1);  // x_{k+1} = x_k +
alpha_k p_k
        cblas_daxpy(n, -alpha, t, 1, r, 1); // r_{k+1} = r_k -
alpha_k A p_k
        eps = cblas_dnrm2(n, r, 1);
        cout << "Iteration " << k << ": eps=" << eps << endl;
        if (eps < tol*tol) {
            break;
        }
        beta = cblas_ddot(n, r, 1, r, 1) / beta;
        cblas_dcopy(n, r, 1, t, 1);
        cblas_daxpy(n, beta, p, 1, t, 1);
        cblas_dcopy(n, t, 1, p, 1);
        k++;
    } while (k < 5000); // Set a maximum number of iterations
    delete[] r;
    delete[] p;
    delete[] t;
}
int main() {
    const int    n   = 8;
    double* M = new double[n*n];
    double* A = new double[n*n];
    double* x = new double[n];
    double* y = new double[n];
    double* b = new double[n];
    srand(time(0));
    // Populate M and x
    // M is symmetric
    FillMatrix(n, M);
    FillVector(n, x);
    // Compute A = M^T M
    cblas_dsyrk(CblasRowMajor, CblasUpper, CblasNoTrans,
                n, n, 1.0, M, n, 0.0, A, n);
    // Print matrices
    cout << "Matrix M:" << endl;
    PrintMatrix(n, M);
    cout << "Symmetric positive A = M*M:" << endl;
    PrintMatrix(n, A);
    cout << "Initial vector:" << endl;
    PrintVector(n, x);
    // Compute a RHS b = A*x
    cblas_dsymv(CblasRowMajor, CblasUpper, n, 1.0, A, n,
                x, 1, 0.0, b, 1);
    // Solve system using the Conjugate Gradient method
    cout << "Solving linear system:" << endl;
    SolveConjugateGradient(n, A, b, y);
    cout << "Solution vector:" << endl;
    PrintVector(n, y);
    // Compare the result y with the original vector x
    cblas_daxpy(n, -1.0, x, 1, y, 1);
    cout << "Error: " << cblas_dnrm2(n, y, 1) << endl;
    delete[] M;
    delete[] A;
    delete[] x;
    delete[] y;
    delete[] b;
    return 0;
}
```

①

cblas_dcopy (const int N, — # of elements to be copied

const double* X, — pointer to the 1st element of the array,

const int incX, — increment between elements
- incX = 1 : put in a row;
- incX = -1 : put in transversed in reverse order.

const double* Y — pointer to the 1st element of the destination array.

const int incY)

② cblas_dsymv (CblasRowMajor

$r = \alpha A x + \beta r$

Uplo, ⇒ CblasUpper

N — # of rows or columns

alpha,

*A

lda, — lda = # columns for row-major; lda = # rows for column-major.

*X,

incX =1

beta

*Y, — name of result vector

incY) =1

α: scalar

A: matrix

x: vector.

β: scalar

r:

③ cblas_ddot (N — # elements in the vector.

X — first vector.

incX =1

Y — second vector.

incY) =1

dot ← $X^T y$.

cblas_daxpy (N

α

X

incX

Y

incY)

$y \leftarrow \alpha x + y$.